

VU Research Portal

PARL: Poisoning Attacks Against Reinforcement Learning-based Recommender Systems

Du, Linkang; Yuan, Quan; Chen, Min; Sun, Mingyang; Cheng, Peng; Chen, Jiming; Zhang, Zhikun

published in
ASIA CCS '24
2024

DOI (link to publisher)
[10.1145/3634737.3637660](https://doi.org/10.1145/3634737.3637660)

document version
Publisher's PDF, also known as Version of record

document license
Article 25fa Dutch Copyright Act

[Link to publication in VU Research Portal](#)

citation for published version (APA)

Du, L., Yuan, Q., Chen, M., Sun, M., Cheng, P., Chen, J., & Zhang, Z. (2024). PARL: Poisoning Attacks Against Reinforcement Learning-based Recommender Systems. In *ASIA CCS '24: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (pp. 1331-1344). ACM.
<https://doi.org/10.1145/3634737.3637660>

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal

Take down policy

If you believe that this document breaches copyright please contact us providing details, and we will remove access to the work immediately and investigate your claim.

E-mail address:
vuresearchportal.ub@vu.nl



PARL: Poisoning Attacks Against Reinforcement Learning-based Recommender Systems

Linkang Du*
Zhejiang University
linkangd@gmail.com

Quan Yuan*
Zhejiang University
yq21@zju.edu.cn

Min Chen
CISPA Helmholtz Center for
Information Security
min.chen@cispa.de

Mingyang Sun
Zhejiang University
mingyangsun@zju.edu.cn

Peng Cheng
Zhejiang University
lunarheart@zju.edu.cn

Jiming Chen
Zhejiang University &
Hangzhou Dianzi University
cjm@zju.edu.cn

Zhikun Zhang[†]
Zhejiang University
zhangzhk@zju.edu.cn

ABSTRACT

Recommender systems predict and suggest relevant options to users in various domains, such as e-commerce, streaming services, and social media. Recently, deep reinforcement learning (DRL)-based recommendation systems have become increasingly popular in academics and industry since DRL can characterize the long-term interaction between the system and users to achieve a better recommendation experience, *e.g.*, Netflix, Spotify, Google, and YouTube.

This paper demonstrates that an adversary can manipulate the DRL-based recommender system by injecting carefully designed user-system interaction records. The poisoning attack against the DRL-based recommender system is formulated as a non-convex integer programming problem. To solve the problem, we proposed a three-phase mechanism (called PARL) to maximize the hit ratio (the proportion of recommendations that result in actual user interactions, such as clicks, purchases, or other relevant actions) while avoiding easy detection. The core idea of PARL is to improve the ranking of the target item while fixing the rankings of other items. Considering the sequential decision-making characteristics of DRL, PARL rearranges the items' order of the fake users to mimic the normal users' sequential features, an aspect usually overlooked in existing work. Our experiments on three real-world datasets demonstrate the effectiveness of PARL and better concealment against the detection techniques. PARL is open-sourced at <https://github.com/PARL-RS/PARL>.

*Both authors contributed equally to this research.

[†]Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASIA CCS '24, July 1–5, 2024, Singapore, Singapore

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0482-6/24/07

<https://doi.org/10.1145/3634737.3637660>

CCS CONCEPTS

• Security and privacy; • Computing methodologies → Machine learning;

KEYWORDS

Poisoning Attack; Recommender System; Reinforcement Learning

ACM Reference Format:

Linkang Du, Quan Yuan, Min Chen, Mingyang Sun, Peng Cheng, Jiming Chen, and Zhikun Zhang. 2024. PARL: Poisoning Attacks Against Reinforcement Learning-based Recommender Systems. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '24)*, July 1–5, 2024, Singapore, Singapore. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3634737.3637660>

1 INTRODUCTION

Recommender systems are a type of recommendation engine that tailors its suggestions to the specific preferences and behaviors of each user [43]. Instead of offering the same set of recommendations to all users, personalized recommender systems use historical data such as past purchases, ratings, or clicks to create a unique profile for each user, which is then used to make personalized recommendations. Recently, deep reinforcement learning (DRL) has been used increasingly in recommender systems since it can well model the interactions between users and systems as a sequential decision-making process [9, 11, 16, 32]. For instance, YouTube uses the DRL-based recommender system to recommend video content to customers [8]. Netflix utilizes DRL-based recommender systems to recommend movies and TV shows to the users [1].

However, recommender systems are not secure and can be fooled by malicious attackers who want to promote or demote certain items. These attackers can create fake users and ratings and inject them into the training data of the recommender system, *i.e.*, *data poisoning attack* [14, 15, 56, 60]. Based on the reports about CHEQ [49] and Facebook [18], there usually exist more than 5% of fake users in real-world applications. Data poisoning attacks can seriously affect recommender systems' trustworthiness and users'

opinions. For example, an attacker can manipulate a news recommender system to show biased or fake news to users and influence their views. Existing data poisoning attacks mainly aim at the traditional recommender systems, *e.g.*, association-rule-based [59], graph-based [15, 39], matrix-factorization-based [14, 26], and deep neural network (DNN)-based [22, 45] recommender systems. However, the existing attacks are inapplicable to DRL-based recommender systems because they are mainly optimized for the one-step recommendation instead of the sequential recommendation scenes. To ensure the security and reliability of DRL-based recommender systems, it is crucial to investigate the potential vulnerabilities of these systems to data poisoning attacks.

Our Contributions. In this paper, we propose a novel data poisoning attack optimized for DRL-based recommender systems. The attacker aims to control the target item recommended to many users. Due to limited resources, the attacker injects a small number of fake users to rate multiple items, including the target and non-target items, to avoid detection. The challenge lies in maximizing the attack’s effect while minimizing the chance of being detected. Furthermore, when constructing the records of the fake user, we consider the sequential decision characteristics in DRL scenarios, which are barely considered in existing work.

We can formulate the attack as an optimization problem yet face two challenges when solving the optimization problem. 1) The selection space of items is vast, *i.e.*, the output space of the DRL model; 2) the protracted length of the users’ records, *i.e.*, the times of interactions between the recommender system and the users. Therefore, when the number of items is fixed, the search space for the solution will grow exponentially with the increase of the sequence length. We designed a three-phase mechanism named PARL to solve the optimization problem approximately. Specifically, the attacker generates two surrogate DRL models for the first challenge, *i.e.*, two *poison models*. The attacker can utilize these poison models to predict rating scores for each fake user and then leverage these scores to select the filler items that maximize the hit ratio of the target item. For the second challenge, Transformer [50] is adopted to learn the relationships between different parts of users’ sequential records. The attacker uses the Transformer model to sort the fake users’ items, minimizing the differences in sequence characteristics between the fake and the normal users.

According to the results, PARL is highly successful in promoting target items and outperforms baseline attacks considerably in a white-box setting. Including only 1% fake users, the hit ratio increased 70× on the ML-1M dataset compared to the normal case. With a mere 0.5% of fake users, PARL obtains a hit ratio of 0.1526 for the target items on the Pantry dataset, 3.8× greater than the best hit ratio achieved by the existing attacks. We also examine the performance of PARL under the partial knowledge scenario, where PARL remains effective and significantly outperforms the benchmark attacks. For instance, when the attacker only knows 30% of the rating records of normal users, PARL yields a hit ratio of 0.1795 for the target item by introducing 1% of fake users on the Pantry dataset, which is at least 1.8× greater than the hit ratio of the competitors. Also, PARL can be transferred to structure-unknown DRL-based recommender systems. Specifically, even if we are unaware of the precise neural network architecture employed by the

recommender system, PARL still increases the hit ratio of the target items by 9× compared to the normal case when injecting 0.5% of fake users into the Pantry dataset.

Furthermore, we evaluate the PARL’s robustness against the potential detection. Firstly, we investigate the identification of fake users through statistical analysis of their rating patterns. The fundamental motivation for this approach derives from the underlying recognition that fake users demonstrate rating patterns that are statistically distinct from those of normal users, attributable to their reliance on predetermined rules. Precisely, we extract multiple features from the ratings of each user, which are subsequently employed to train an SVM classifier to identify potential fake users. The service provider eliminates the detected fake users before training the recommender system. The experimental results demonstrate the effectiveness of the detection based on rating patterns. However, PARL remains more effective than the existing work, where 20% fake users generated by PARL are incorrectly identified as normal users. Secondly, we leverage a convolutional neural network (CNN) classifier to distinguish the normal and the fake users based on the sequential features. The experimental results show that the fake users’ records are harder to detect after the order arrangement.

In summary, our contributions are three-fold:

- To the best of our knowledge, PARL is the first poisoning attack against the DRL-based recommender systems. We formulate the poisoning attack as a non-convex integer programming problem and utilize three phases to optimize its attacking capability.
- We demonstrate the effectiveness of PARL on three real-world datasets. We also systematically analyze various experimental factors, *i.e.*, the number of fake users, the scale of items, the number of filler items, the partial knowledge, and the transferability of PARL, and summarize a series of critical observations.
- In the end, we explore two types of methods for defending against the proposed poisoning attacks, *i.e.*, rating score-based detection and fake user detection based on sequence features. We found that PARL is robust to the rating score-based detection. However, detecting fake users based on sequence features is a valid way to defend against PARL. We show that recovering the sequence of the records can improve the concealment of fake users, which has not drawn enough attention in existing work.

2 PRELIMINARIES

2.1 DRL-based Recommender Systems

The recommendation procedure between the users and the recommender system can be formulated into a Markov decision process (MDP) problem [69]. At each time step, the recommender system takes an action $a \in \mathcal{A}$ based on the user state $s \in \mathcal{S}$, and receives a reward $R(s, a)$. Then, the state is updated to s_{t+1} with $p(s_{t+1}|s_t, a_t)$. Following [31], the elements of an MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ are defined as follows:

- **States $\mathcal{S}$.** A state s records a set of items recommended and liked by the user (*i.e.*, positive feedback).
- **Actions $\mathcal{A}$.** An action a is to select an item available in the item space.
- **Transitions $\mathcal{P}$.** $p(s_{t+1}|s_t, a_t)$ represents the state transition once the recommender system executes action a .
- **Reward $\mathcal{R}$.** After conducting the action a for the user state s , the recommender receives immediate reward $R(s, a)$.
- **Discount rate γ .** This factor adjusts the model's attention to future rewards.

Figure 1 illustrates the user-system interactions in DRL-based recommender systems. The current state s_t represents a set of items bought by the users recently. The recommender system will take action a_t based on the state s_t , *i.e.* recommend an item. Then, the user rates the item, and the recommender system receives the reward r_t . The next state s_{t+1} will be updated accordingly.

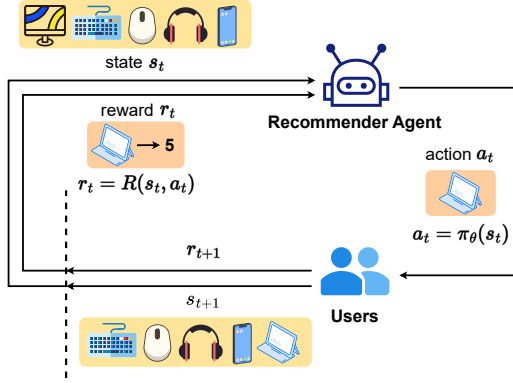


Figure 1: A running example of the interacting process between user and DRL-based recommender systems.

DRL-based recommender systems aim to uncover an optimal recommendation strategy with high user satisfaction. Regarding optimization strategies, the existing framework of DRL can be categorized into three classes [1].

Value-based RL models. Q-learning [53] is a prominent example of a value-based RL model. It aims to find the optimal policy by maximizing the expected value of the cumulative reward over subsequent steps. Q-learning employs a function to evaluate the quality of a state-action pair: $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. In the basic Q-learning approach, Q is initially defined as a tabular structure, known as the Q-table, with random values before the learning process starts. At each time step t , the model selects an action a_t , receives a reward r_t , transitions to a new state s_{t+1} (which depends on both the previous state s_t and the chosen action a_t), and then updates the values in Q . The main steps of the algorithm are as follows:

$$Q^{\text{new}} \leftarrow Q + \alpha \cdot \left(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right), \quad (1)$$

where $\alpha \in (0, 1]$ represents the learning rate, and a larger α leads to more rapid changes in Q^{new} . The equation 1 illustrates how Q-learning combines the old Q values with new information using a

weighted average, known as value iteration update. When the state and action spaces become extremely large, the basic Q-learning approach struggles to maintain and store an extensive Q-table. To overcome this challenge, Deep Q-Learning Network (DQN) [36] incorporates Deep Neural Networks (DNNs) to approximate the values in the Q-table. Leveraging the remarkable ability of neural networks to learn high-dimensional feature representations and perform function approximation, DQN has demonstrated expertise in Go [47] and Atari games [36].

Policy-based RL models. In contrast to approximating the Q-table, policy-based RL models directly optimize action predictions. For instance, REINFORCE [55] updates a neural network using gradient estimation. Recent policy-based RL models often employ DNNs, such as multilayer perceptron [57] or recurrent neural networks [64], to instantiate the policy network π . The objective function of the policy network is defined as the expected total discounted rewards in a game:

$$J(\theta) = \mathbb{E}(s_t, a_t) \sim \pi_\theta \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]. \quad (2)$$

From [24], the policy gradient can be expressed as $\nabla_\theta J(\theta) = \mathbb{E} \pi_\theta [\nabla_\theta \log \pi_\theta(s, a) v_t]$, where v_t represents an unbiased sample of $Q_{\pi_\theta}(s, a)$, indicating the long-term reward of the MDP problem, and π_θ corresponds to the learned policy network.

Actor-Critic RL models. The actor-critic RL model [35] employs two distinct components, namely the actor and the critic, to represent the policy and the action-value function separately.

The actor is responsible for determining the optimal action, while the critic evaluates and scores the actor's chosen action. The actor learns using the policy gradient approach, while the critic assesses the actor's action by estimating the action-value function.

The actor-critic RL model addresses issues such as gradient variance and expedites the convergence process by utilizing the action-value function to facilitate policy updates. Consequently, the actor-critic RL model features two objective functions for the actor and critic networks. For the actor, the objective function bears a resemblance to Equation 2 but employs $A_{\pi_\theta}(s, a)$ instead of $Q_{\pi_\theta}(s, a)$.

$$A_{\pi_\theta}(s, a) = Q_{\pi_\theta}(s, a) - V_{\pi_\theta}(s), \quad (3)$$

where $V_{\pi_\theta}(s)$ is the average value of state s . When $A_{\pi_\theta}(s, a) = 0$, it implies that there is no additional reward to be gained by adjusting the action probabilities, resulting in the gradients of the actor being set to zero. The objective of the critic is to minimize the discrepancy between the estimated and true action-value functions. Two representative methods based on this framework are Proximal Policy Optimization (PPO) [20, 46] and Deep Deterministic Policy Gradient (DDPG) [27], which deliver comparable or superior performance compared to state-of-the-art methods while being more straightforward to implement and fine-tune.

2.2 Data Poisoning Attacks

Poisoning attacks against recommender systems refer to deliberate attempts to manipulate the behavior of these systems by injecting fake users and their associated ratings into the system's training dataset, thereby compromising the integrity of the learning process. These attacks can significantly negatively impact recommender systems' performance, fairness, and trustworthiness. For example,

poisoning attacks can be used to manipulate the recommendations for malicious purposes. The attackers might try to promote their products, services, or content, or they want to attempt to mislead users by injecting fake information. The existing data poisoning attacks against recommender systems can be categorized into two classes by whether they are optimized for a specific system.

System-untargeted Attacks. These attacks do not consider the specific algorithm used by the recommender system and are generally less effective. Examples of algorithm-agnostic attacks include random attacks and bandwagon attacks. In random attacks [23, 38], the attacker registers fake users in the web service associated with the recommender system and assigns random rating scores to a subset of items from the entire item set. These attacks lack sophistication and often have limited effectiveness. Bandwagon attack [23, 42] involves selecting highly popular items in the dataset for fake users. By manipulating the ratings of popular items, the attacker aims to bias the recommendations toward those items.

System-targeted Attacks. These attacks are optimized for a specific type of recommender system. Thus, they are typically more effective than system-untargeted attacks, such as association-rule-based [59], graph-based [15, 39], matrix-factorization-based [14, 26], and DNN-based [22, 45] recommender systems. The attackers can optimize data poisoning attacks specifically for this type of system, aiming to modify the learned latent factors to influence the recommendations [26].

3 PROBLEM STATEMENT

3.1 Threat Model

Attacker’s Goal. The attacker attempts to maximize the likelihood of the target item being recommended when the DRL-based recommender system interacts with normal users, *i.e.*, poisoning effectiveness. On the other hand, the attacker should keep the relative ranking of the remaining items to minimize the impact of improving the target item on the normal behavior of the DRL-based recommender system, *i.e.*, poisoning stealthiness.

Attacker’s Capabilities. Since the users’ ratings are often publicly accessible in popular recommender systems like Amazon and Taobao, we assume that the attacker can access the user-item rating records between the recommender system and the users. The attacker can employ a web crawler to gather these ratings. Furthermore, in Section 5.3, we demonstrate that PARL is still effective with partial records (30%). Regarding the DRL model details of the recommender system, we examine both white-box and gray-box access to the DRL model. Initially, we present the performance of PARL when the attacker possesses complete knowledge of the DRL model in Section 5.2. Subsequently, we verify that PARL can successfully transfer its effectiveness across different DRL models in Section 5.4.

3.2 Attack Formulation

We assume that the attacker can inject M fake records, noted as D_p , into the DRL-based recommender system. To achieve this, the attacker can generate and publish some bogus users’ records or register some accounts to interact with the recommender system. During the generation of the sequential records, the attacker needs

to pack some filler items to avoid easy detection. To maximize the recommending probability of the target item a^* , the attacker should build an optimal $\hat{D}_p$, a set of filler items with proper rating scores. Besides, the selected filler items should prevent the fake user from being detected. Inspired by the existing work [2, 48], the optimization problem can be formulated by following:

$$\begin{aligned} \hat{D}_p &\in \operatorname{argmax}_{D_p} Q_{\hat{\theta}}(s, a^*) \\ \text{subject to } \hat{\theta} &\in \operatorname{argmin}_{\theta} \mathcal{L}_{Q_{\theta}}(D \cup \hat{D}_p, \theta), \end{aligned} \quad (4)$$

where θ is the trainable parameters of the policy network, and D represents the normal users’ records.

For ease of understanding, we use Φ_i to denote the series of items of the i -th fake user, and the $\mathbf{r}_i$ is the corresponding score for the items. D_p can be written as $\{\Phi_i, \mathbf{r}_i | i \in \{1, 2, \dots, M\}\}$. Without loss of generality, we set H as the total number of potential items and $N + 1$ as the maximum number of interactions between the recommender system and the users. Each fake user has a single target item and a maximum of N filler items. We adopt a widely-used rating strategy for the recommender system [14, 15, 22], *i.e.*, ranking items with integer-valued scores from the preset set $\{1, 2, \dots, r_{\max}\}$. Since the sequential recommendation-making process of DRL [69], compared with the existing work [14, 15, 22], the attacker should maintain the sequential property of the fake users’ records as that of the normal users. We set $\odot_D$ to represent the sequential features of the normal users’ records. Thus, the generated D_p should satisfy the additional constraints.

$$\begin{aligned} \|\Phi_i\|_0 &\leq (N + 1), \\ \mathbf{r}_i[j] &\in \{1, 2, \dots, r_{\max}\}, \\ \Phi_i[j] &\in \{1, 2, \dots, H\}, \\ \Phi_i &\sim \odot_D, \end{aligned}$$

where $i \in \{1, 2, \dots, M\}$ and $j \in \{1, 2, \dots, \|\Phi_i\|_0\}$. The first constraint limits the maximum volume of filler items allocated to each fake user. The second and third constraints indicate that fake records should satisfy the normal record format of the recommender system. The last one means the fake records should follow the sequential features of the normal users’ records. Since finding an optimal solution for a nonlinear integer programming problem is challenging, we design a three-phase mechanism to solve the problem approximately.

4 METHODOLOGY

4.1 Intuition

The solution of Equation 4 consists of two parts: The *items* and the *scores*. The adversary first initializes each fake user with the target item. Then, the adversary selects the filler items from the entire set of items. In general, the filler items are not duplicates. After fixing the target item, each filler item has two possible situations relative to the target item, *i.e.*, the initially recommended ranking in dataset D is higher or lower than the target item. Combined with the scores assigned for the filler items, *i.e.*, the assigned score is higher or lower than the scores in D , the filler item has four possible impacts on the target item. Table 1 indicates that the filler item promotes

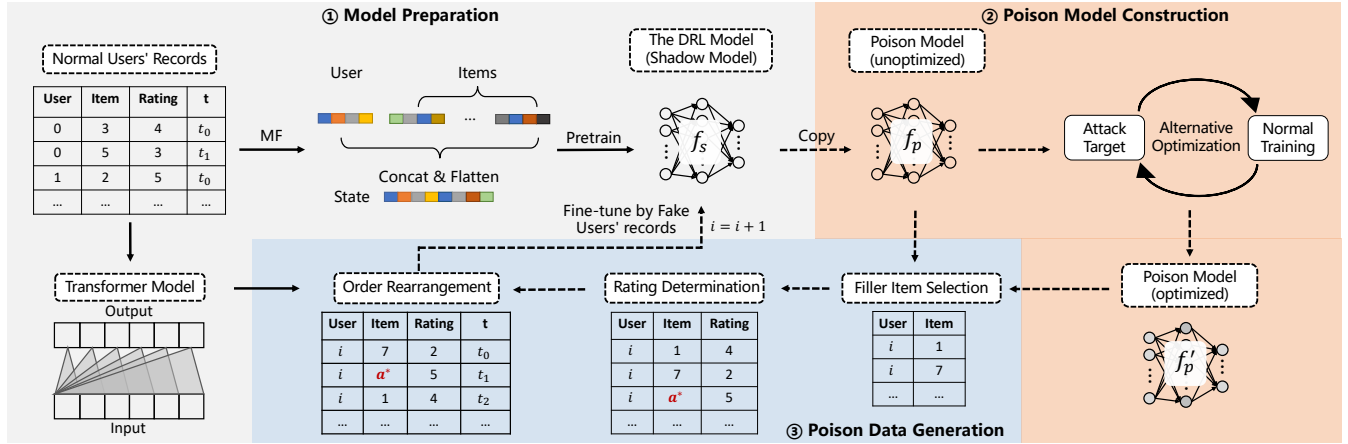


Figure 2: PARL overview. PARL comprises three phases: Model Preparation, Poison Model Construction, and Poison Data Generation. In the model preparation phase, the adversary aims to train two models for the following steps, i.e., a shadow model and a Transformer model. In the poison model construction phase, PARL alternatively optimizes the unoptimized poison model (i.e., a copy of the shadow model) by two loss functions and obtains the optimized poison model. In the poison data generation phase, PARL generates a series of items for a fake user based on three steps, i.e., filler item selection, rating determination, and order rearrangement.

the target item when it has a higher initial recommendation ranking yet receives a low score. In this case, the filler item's ranking will decrease compared to the target item. However, assigning low scores to high-ranking items is an abnormal behavior that is easy to detect. In other cases, the filler items do not positively influence the optimization goal.

Inspired by the above observations, the core idea of PARL is to improve the ranking of the target item while fixing the ranks of other items, aiming to achieve a balance between hit ratio and avoiding detection. Thus, PARL is designed from the following two aspects aligning with the insight: 1) Picking out the filler items whose ranks are not sensitive to the rank raising of the target item; 2) Maintaining the sequential feature of the fake users consistent with that of the normal users.

4.2 Workflow

For ease of understanding, we use D to represent the clean data from the normal users. PARL generates and injects the poisoning data D_p into D to launch the attack. Figure 2 illustrates the workflow of the proposed method, where PARL is initialized by Phase 1 and loop executes the following two phases to generate fake users. We also provide the pseudocode of PARL in algorithm 1.

Phase 1: Model Preparation (MP). The adversary trains two models using clean data D . The Transformer model captures dependencies and relationships between items in sequential records. This makes it particularly suitable for rearranging the order of items in Phase 3. Additionally, the adversary employs the normal users' records to train a shadow model, which is used to mimic the behaviors of the target recommender system. Once the pre-training is completed, the adversary uses the shadow model as a substitute for the target model to generate fake user data. From the second round, the adversary only fine-tunes the shadow model using the

Table 1: The four impacts of filler item on the target item.

Case	Original Rank	Assigned Score	Impacts
1	Higher	Higher	Negative
2	Higher	Lower	Positive
3	Lower	Higher	Negative
4	Lower	Lower	None

records of fake users instead of training it from scratch to reduce costs.

Phase 2: Poison Model Construction (PMC). The adversary duplicates the latest shadow model (unoptimized poison model) from Phase 1 and optimizes it to construct an optimized poison model. The adversary's objective is to promote the ranking of the target item among users who do not already have it in their records. Additionally, the poison model needs to maintain normal recommendations for the users, which is guided by the normal training loss, ensuring the model predicts desirable items to maximize the cumulative rating score. By iteratively optimizing the poison model, the attacker strikes a balance between maintaining the model's validity and achieving the attack objective. After optimization, the attacker generates a poison model that resembles a normal recommender system while fulfilling its attack goal. The optimized poison model is utilized to select items for fake users.

Phase 3: Poison Data Generation (PDG). After the above two phases, the adversary obtains two versions of the poison models, i.e., the unoptimized and optimized models. Recalling the analysis in Section 4.1, PARL is designed to improve the ranking of the target item while fixing the rankings of other items. PARL randomly selects the records from the users whose records do not contain

the target item. The initial state is built by combining a user ID and several items and converted into an embedding form. Then, PARL queries the poison models with the initial state's embedding to gain the predicted values for all items. By comparing the predicted values from the unoptimized and optimized models, PARL selects the filler items with smaller prediction changes.

In the rating determination step, PARL seeks to maximize the recommendation probability while concealing the fake user. Instead of assigning a fixed maximum score to the target item, PARL scores the target item based on the distribution of scores in the target dataset. Since items with low frequency tend to have extreme scores, *e.g.*, all scores are one or r_{max} , the scoring distribution is determined by selecting items with frequencies similar to or higher than the poisoning ratio and analyzing their rating records. PARL finds the item with the highest percentage of r_{max} and assigns a score to the target item based on its rating distribution. For filler items, PARL calculates mean and variance from normal user ratings and sampling scores from a normal distribution.

In the order rearrangement step, PARL ensures the sequential feature of fake items aligns with normal users' records. PARL achieves this by using a Transformer model to predict the next item based on the previous fake items, thus restoring the sequential property.

After the above three phases, PARL generates a series of items for one fake user. To produce more fake users, PARL repeatedly conducts Phase 2 and Phase 3. In the first round, the shadow model is trained on normal users' records D . In the following rounds, PARL fine-tunes the shadow model with the fake users' records to simulate attack results and then executes Phases 2 and 3.

4.3 The Optimization of Poison Model

The adversary duplicates the latest shadow model (unoptimized poison model) from Phase 1 of Figure 2, then adjusts its parameters to construct the optimized poison model. On the one hand, the adversary needs to promote the ranking of the target item by minimizing $\mathcal{L}_1$. Since the target item exists in the records of the partial normal users, the adversary only needs to promote the target item among the remaining users. We use D_{tar} to demonstrate the users whose records contain the target item, and $D_{\bar{tar}}$ is the other users in D , where $D = D_{tar} \cup D_{\bar{tar}}$ and $\emptyset = D_{tar} \cap D_{\bar{tar}}$.

$$\mathcal{L}_1 = \max \left(\max_a Q(s, a) - Q(s, a^*), 0 \right), \quad (5)$$

where a^* represents recommending the target item. On the other hand, the goal of Equation 6 is to guide the poison model Q to maintain the normal recommendation.

$$\mathcal{L}_2 = \mathbb{E}_D \left[\left(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right)^2 \right], \quad (6)$$

where r_t is the rating score for the recommendation a_t . The recommender system maximizes the cumulative rating score by predicting the desirable items for the users. When minimizing $\mathcal{L}_2$, the adversary utilizes the normal users' records D .

By iteratively optimizing the poisoning model's parameters using Equation 5 and Equation 6, the adversary finds an equilibrium between preserving the model's validity and accomplishing the attack objective. The optimized poisoned model closely mimics the model trained under regular conditions while effectively fulfilling the attack goal.

Algorithm 1: Workflow of PARL

Input: Dataset D , target item a^* , the number of fake users M , the number of available items H , the number of filler items N

Output: M fake users

```

1 // Phase 1: Model Preparation
2 Train the Transformer model and the shadow model.
3 for fake user  $i$  in  $\{1, 2, \dots, M\}$  do
4   if  $i > 1$  then
5     Use the fake user(s) to fine-tune the shadow model.
6   end
7 // Phase 2: Poison Model Construction
8 Obtain the optimized poison model  $f'_p$  by alternative
  optimization with Equation 5 and Equation 6.
9 // Phase 3: Poison Data Generation
10 // Phase 3.1: Filler Item Selection
11 // Select the filler items with predicted values that
  change smaller between the unoptimized and the
  optimized poison model.
12  $\{a_j | j \in \{1, 2, \dots, N\}\} =$ 
   $\operatorname{argmin}_{a \in \{1, 2, \dots, H\}; s \sim D_{\bar{tar}}} \|f_p(s, a) - f'_p(s, a)\|$ .
13 // Phase 3.2: Rating Determination
14 Assign the rating score to the target item  $a^*$  based on
  the score distribution of the item with the highest
  percentage of  $r_{max}$ .
15 for filler item  $j$  in  $\{1, 2, \dots, N\}$  do
16   Generate rating scores for the filler item  $j$  of user  $i$ :
17    $r_{ij} \sim \mathcal{N}(\mu_j, \sigma_j^2)$ .
18 end
19 // Phase 3.3: Order Rearrangement
20 Restore the sequential property of the  $i$ -th fake user's
  items by the Transformer model in Phase 1.
21 end
22 return  $M$  fake users

```

4.4 The Details of PDG

Filler Item Selection. Recalling the analysis in Section 4.1, PARL is designed to improve the ranking of the target item while fixing the rankings of other items. In this step, the adversary utilizes two versions of the poison models, *i.e.*, the unoptimized and optimized models. PARL randomly select the users' records from $D_{\bar{tar}}$ and construct the state, *i.e.*, the input of the DRL model. The state is built by combining a user ID and several items and converted into an embedding form. Then, PARL queries the poison models with the state to gain the predicted values for items in the following recommendation. By comparing the predicted values of the unoptimized and optimized models, PARL selects the filler items that show more minor changes in predicted values. In practice, as different states influence the predicted values of the poison models, PARL randomly constructs a set of states to query the poison models and selects the items with a smaller sum of absolute variations.

Rating Determination. To maximize the recommendation probability, existing work [14, 15, 22] usually assigns r_{max} for the target

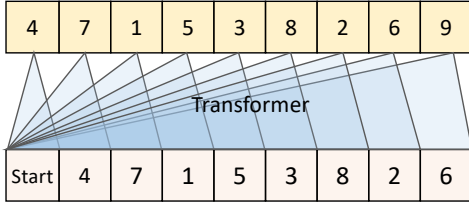


Figure 3: The order rearrangement for the fake user's items.

item. However, when the defender knows the poisoning method, the fake user constructed by this assignment method can be easily detected. To improve the concealment of fake users, PARL hopes that the score of the target item conforms to the distribution of the target dataset D . We score the target item based on the scoring distribution of items with similar and higher frequencies in D . For example, when the poisoning rate is 10%, and the number of the items' records is 1000, PARL will select the items whose frequencies are near or greater than $1000 \times 10\% = 100$ and obtain every item's scoring distribution among the records. PARL searches the item with the highest r_{max} ratio η of its rating records. Then, PARL assigns r_{max} to the target item until the r_{max} ratio of the target item reaches η . We do not consider the scores less than $(r_{max} - 1)$, since $(r_{max} - 2)$ may not be a positive recommendation, such as $(r_{max} - 2)$ is the lowest score when $r_{max} = 3$. We adopt a similar strategy for the filler items to the existing work [14, 15, 22]. Specifically, for the filler item j , PARL first calculates the mean μ_j and the variance σ_j^2 by using the rating scores from the normal users. Then, PARL samples r_j from the normal distribution $\mathcal{N}(\mu_j, \sigma_j^2)$.

Order Rearrangement. PARL restores the sequential order of the fake items using the Transformer model trained in Phase 1. We demonstrate this process with a running example in Figure 3. Initially, PARL randomly chooses one item (e.g., item 4) and combines it with the start signal. It then queries the Transformer model to predict the second item (e.g., item 7). Next, PARL takes item 4 and item 7 as input and gains item 1. PARL continues this procedure by using the sorted items as input and obtaining the next item until all fake items have been traversed.

5 EVALUATION

5.1 Experimental Setup

Datasets. We use three real-world datasets to evaluate the performance of PARL. Table 2 shows the basic information of the datasets. To ensure the state updates normally, we pre-process the datasets. Specifically, we remove the users with a low number of ratings and keep the users with at least ten positive ratings. The information about the pre-processed datasets is as follows.

- **Pantry [40]** is a review dataset for prime pantry in Amazon, which includes 2,945 users, 7,035 items, and 57,911 ratings ranging from 1 to 5.
- **Yahoo [34]** contains the user ratings of music collected by Yahoo. It comprises 4,451 users, 1,000 items, and 124,344 ratings ranging from 1 to 5.

Table 2: Dataset Information.

Dataset	Users	Items	Ratings	Type
Pantry [40]	2,945	7,035	57,911	Shopping
Yahoo [34]	4,451	1,000	124,344	Music
ML-1M [19]	5,950	3,685	840,664	Movie

- **ML-1M [19]** is a classic movie dataset comprising 5,950 users, 3,685 items, and 840,664 ratings ranging from 1 to 5.

Target Recommender System. The three frameworks discussed in the Section 2.1 are widely adopted in existing DRL-based recommender systems [30]. Since the real-world recommender systems usually select recommended items from a category-based item set, the value-based RL models (i.e., Q-learning and DQN) fit these discrete decision scenarios. They are popularly adopted in existing DRL-based recommender systems [1]. Thus, our evaluation is mainly conducted on the DQN-based recommender system. The recommender system utilizes a DQN model to train and recommend the items. The episode length T is fixed for each user. The state is the 100-dimensional embeddings of the user and the current n_s positive items (ratings are 4 and 5). The reward is normalized to $[-1, 1]$.

Competitors. We compare PARL to several existing poisoning attacks. An attacker injects M fake users into the recommender system in all these attacks. Every fake user gives the rating score r_{max} or $(r_{max} - 1)$ to the target item and gives certain rating scores to N filler items. The competitors use various strategies to choose the filler items and assign scores to them.

- **Random Attack [23, 38]** randomly chooses N filler items for each fake user. Then, for each filler item, the attacker samples a number from the normal distribution on the initial user-item rating matrix and dives it into a rating score.
- **Bandwagon Attack [23, 42]** considers the items' popularity, i.e., the average score of the item. For each fake user, the attacker selects $N \times 10\%$ items from the set of 10% items with the highest popularity and $N \times 90\%$ items among the remaining items.
- **MF Attack [26]** initializes a user-item rating matrix based on normal user records and implements an optimized poisoning attack. Here, we deploy the PGA attack in [26] as one of the baselines to compare with PARL.
- **DNN Attack [22]** regards the target system as a DNN-based recommender system. The optimized model is constructed based on a DNN model.

Evaluation Metrics. We use the hit ratio (HR@K) as the evaluation metric. We calculate whether the target item appears in each user's top- K recommendation lists in 5 consecutive states. If the target item appears, the hit ratio for the user is 1; otherwise, it is 0. Finally, we calculate the average hit ratio for all users as the HR@K. We randomly select ten items as the target items and compute the hit ratio averaged over these target items for all users.

Table 3: Hit Ratio (HR)@10 of the poisoning attacks with different numbers of fake users.

Attack	Attack size								
	Pantry			Yahoo			ML-1M		
	0.5%	1%	2%	0.5%	1%	2%	0.5%	1%	2%
None	0.0118	0.0118	0.0118	0.1858	0.1858	0.1858	0.0001	0.0001	0.0001
Random	0.0189	0.1281	0.2973	0.2289	0.3091	0.4091	0.0002	0.0017	0.0162
Bandwagon	0.0405	0.1625	0.2499	0.2127	0.2824	0.4040	0.0008	0.0012	0.0188
MF	0.0213	0.1913	0.3329	0.2652	0.2992	0.4638	0.0004	0.0005	0.0294
DNN	0.0152	0.0488	0.4014	0.2141	0.2665	0.4683	0.0005	0.0026	0.0141
PARL	0.1526	0.5626	0.8467	0.2992	0.3676	0.4856	0.0006	0.0071	0.0386

Parameter Setting. The default parameter settings for PARL are as follows: $T = 30$, $N = 30$, $K = 10$, $n_s = 5$, and M is the number of fake users, which is 1% of the normal users in the recommender system. We implement PARL with Python 3.8, and all experiments are conducted on a server with 4 NVIDIA GeForce RTX 3090 GPUs, AMD EPYC 7402@2.8GHz, and 128GB memory.

5.2 Effectiveness of PARL

In this section, we conduct experiments under the white-box setting, where the attacker can access the full knowledge of the recommender system. Specifically, the attacker knows the target recommender system’s internal structure, training data, and hyperparameters.

Impact of the Number of Fake Users. Table 3 illustrates the hit ratios of poisoning attacks with different numbers of fake users. In the table, the attack size is the fraction of the number of fake users to that of original normal users. “None” means no poisoning attacks implemented on the target recommender system, MF represents the poisoning attack on the matrix-factorization-based recommender systems, and DNN represents the attack on the deep-neural-network-based recommender systems. 1) We find that as the attack size increases, the hit ratios of all attack methods show an upward trend. Moreover, PARL is especially effective in promoting target items on all datasets. For instance, when injecting 1% fake users into the ML-1M dataset, the hit ratio increased by 70%.

2) PARL significantly outperforms the baseline attacks in most cases. For example, PARL performs better than all competitors under all attack sizes on the Pantry and Yahoo datasets. In particular, for the Pantry dataset, PARL achieves the hit ratio of 0.1526, about 3.8× of the best hit ratio obtained by other attacks. For the ML-1M dataset, PARL achieves higher hit ratios than other methods when the attack size is 1% and 2%, yet performing worse than Bandwagon when the attack size is 0.5%. The reason might be that the average scores of most items in the ML-1M dataset are low, and a 0.5% attack size is not enough to demonstrate the advantages of PARL. Nonetheless, PARL still shows a competitive performance compared with other methods.

3) The attack performance varies on different datasets. We speculate that the rating distributions on the items are different in

datasets. Figure 4 shows the items’ frequency and rating distributions of different datasets. The frequency distribution of the three datasets is similar, *i.e.*, the long-tailed distribution. However, the rating distributions of the datasets are distinct. The number of items with high ratings for the Pantry dataset is significantly more significant than the other two. The selected filler items significantly impact the final attack performance, as inappropriate filler items will have an inhibitory effect on improving target items. PARL can avoid selecting these types of items and instead choose the items that hinder the progress of the target item. Accordingly, PARL demonstrates excellent advantages compared to other methods. For the Yahoo dataset, the average ratings of most items are concentrated above 3. Therefore, the adversary achieves a better attack effect when injecting only a small amount of fake users. For the ML-1M dataset, the number of items with low ratings is more significant than that of the Pantry and Yahoo datasets. Therefore, the attack performance of all methods decays on the ML-1M dataset.

Impact of the Number of Recommended Items. Table 4 shows the hit ratios of the attacks with different numbers of recommending items to the users, *i.e.*, the parameter K . The attack size is set to 1%.

We have two observations here. First, the hit ratios of all methods tend to increase with K , and PARL still shows an advantage over other attacks on the three datasets. Second, when K is smaller, the increasing hit ratio of PARL is more significant than the existing attacks. For instance, when $K = 5$ and $K = 20$, the hit ratio of PARL improves upon the best existing attacks by 3.8× and 2.5× on the Pantry dataset, respectively. Moreover, for the ML-1M dataset, the hit ratio of PARL is 5× better than the best baseline attack when $K = 5$. In contrast, the hit ratio of PARL is 1.4× more than other competitors when $K = 20$. It indicates that PARL ranks the target item in the recommendations higher than other methods. The hit ratio of the DNN attack is low since it selects many items with high average ratings, suppressing the promotion of the target item.

Impact of the Number of Filler Items. Figure 5 shows the results of poisoning attacks with different numbers of filler items (*i.e.*, N) that vary from 20 to 50. We can obtain the following observations. First, PARL performs better than other existing poisoning attacks in all test cases, which further illustrates the effectiveness and robustness of our proposed attack. Second, the hit ratio might not always increase as N increases. On the Pantry and ML-1M datasets,

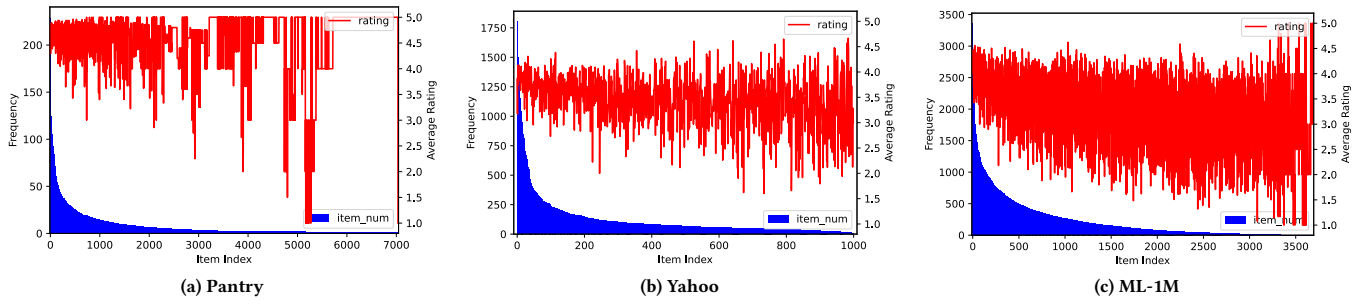


Figure 4: The items' frequency and rating distributions of different datasets.

Table 4: HR@K for different K.

Dataset	Attack	K			
		5	10	15	20
Pantry	None	0.0065	0.0118	0.0198	0.0233
	Random	0.0918	0.1282	0.1568	0.1885
	Bandwagon	0.1107	0.1625	0.2197	0.2528
	MF	0.1310	0.1913	0.2061	0.2321
	DNN	0.0275	0.0489	0.0698	0.0852
	PARL	0.4968	0.5626	0.6098	0.6429
Yahoo	None	0.1104	0.1858	0.2833	0.3353
	Random	0.2172	0.3091	0.3681	0.4193
	Bandwagon	0.1865	0.2824	0.3521	0.3973
	MF	0.1967	0.2993	0.3662	0.4157
	DNN	0.1631	0.2665	0.3349	0.3846
	PARL	0.2774	0.3676	0.4232	0.4630
ML-1M	None	0	0.0001	0.0003	0.0008
	Random	0.0001	0.0017	0.0101	0.0255
	Bandwagon	0.0001	0.0012	0.0081	0.0217
	MF	0.0001	0.0005	0.0034	0.0140
	DNN	0.0003	0.0026	0.0089	0.0231
	PARL	0.0015	0.0071	0.0181	0.0364

the performance of PARL increases first and then decreases with the rise of N . The results illustrate that there is no linear correlation between the attack effectiveness and N , and the relationship between the hit ratio and the number of filler items mainly depends on the datasets.

5.3 Attacks with Partial Knowledge

In this section, we assume that the attacker only has partial knowledge of the dataset, *i.e.*, the attacker knows the partial rating scores of all normal users and generates fake users based on the partial rating scores. Then, we evaluate the attack performance on the whole dataset, which contains all users and rating scores. Table 5

Table 5: HR@10 with a subset of users.

Knowledge level	Attack	Dataset		
		Pantry	Yahoo	ML-1M
30 %	None	0.0104	0.1692	0.0001
	Random	0.0983	0.2744	0.0030
	Bandwagon	0.0425	0.2843	0.0008
	MF	0.0858	0.3047	0.0004
	DNN	0.0131	0.1871	0.0002
	PARL	0.1795	0.3321	0.0091

shows the hit ratios of all methods with 30% partial knowledge on the three datasets. 1) PARL still performs better than other attack methods on all datasets. We speculate that PARL has a more vital generalization ability. For example, PARL's hit ratios with partial knowledge are similar to those of complete knowledge on the Yahoo and ML-1M datasets.

2) The hit ratios of most competitors drop in the case of partial knowledge. For instance, the attack performance decreases in the partial knowledge case compared to the entire knowledge case on the Pantry and Yahoo datasets. The reason is that the attacker may not access the items with high ratings for the Pantry dataset when they only have partial knowledge of the dataset, affecting the selection of filler items.

3) The rating distributions of items and the partial knowledge of the dataset have coupling effects on the hit ratios. The drop in hit ratios in the Pantry dataset is more pronounced than in the Yahoo dataset. Since most average ratings of items are between 3 and 4, the impact of partial knowledge is weakened on the Yahoo dataset. However, the range of the items' average ratings is extensive on the Pantry dataset, and partial knowledge affects the optimization of these attacks compared to the full knowledge case.

5.4 Transferability

To further evaluate the transferability of PARL, we consider the gray-box setting where the attacker only knows the algorithm and the training data the target recommender system uses. In this section, we assume that the attacker generates fake users based on a shadow model, which is different from the internal structure of the

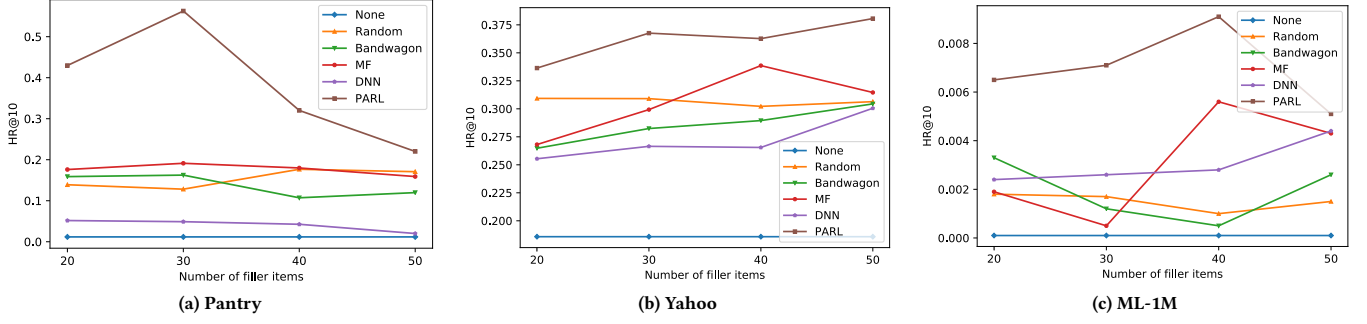


Figure 5: Impact of the number of filler items.

Table 6: HR@10 under the transferability setting.

Dataset	Attack	Attack size		
		0.5%	1%	2%
Pantry	None	0.0102	0.0102	0.0102
	Random	0.0267	0.1278	0.3904
	Bandwagon	0.0511	0.1031	0.3330
	MF	0.0229	0.1645	0.4196
	DNN	0.0456	0.0917	0.5022
	PARL	0.0947	0.3330	0.5124
Yahoo	None	0.1629	0.1629	0.1629
	Random	0.2413	0.3018	0.3736
	Bandwagon	0.2504	0.2873	0.3943
	MF	0.2235	0.2918	0.4021
	DNN	0.2010	0.2598	0.3775
	PARL	0.2918	0.3311	0.3858

target recommender system. Specifically, we change the number of layers and neurons of the neural network to construct a different target recommender system.

Table 6 shows the hit ratios of PARL and the existing attacks. We provide the results on the Pantry and Yahoo datasets. 1) PARL shows the best transferability in most cases. When the attack size is small, e.g., 0.5% and 1%, PARL performs better than other attack methods on the datasets.

2) All attacks can increase the hit ratios of target items effectively. That is, all competitors' hit ratios exceed the baseline by at least 50%. When the attack size is up to 2%, the hit ratios of all competitors exceed 0.33. Especially, PARL achieves a 50× improvement compared to the initial hit ratio on the Pantry dataset.

6 DEFENSE

Service providers can deploy specific fake user detectors for the recommender system to mitigate the impact of potential attacks on the recommender system. In this section, we design two types of

detectors (i.e., rating score-based and sequence-based) to verify the effectiveness of PARL.

6.1 Rating Score Based Detection

Similar to the existing defenses [10, 22, 37] that utilize several statistical features from rating scores to distinguish normal users from fake users, we leverage these features to train the detection classifiers. We extract five features, i.e., RDMA [10], WDMA [37], WDA [37], FMTD [37], and MeanVar [37], for each user from its ratings. The details of these features are described as follows.

- *Rating Deviation from Mean Agreement (RDMA)* [10]. The feature represents the average deviation of a user's rating scores from the mean rating scores of the corresponding items. This computation is performed as follows for a given user u :

$$RDMA_u = \frac{\sum_{i \in r_u} \frac{|\Phi_{ui} - \overline{\Phi^{(i)}}|}{m_i}}{|r_u|},$$

where r_u is the set of items that user u has rated, $|r_u|$ is the number of items in r_u , Φ_{ui} is user u 's ratings score for item i , $\overline{\Phi^{(i)}}$ is the average rating score of item i , and m_i is the total number of ratings for item i in the whole dataset.

- *Weighted Degree of Agreement (WDA)* [37]. The feature represents the numerator of the RDMA feature, and its computation is carried out as follows:

$$WDA_u = \sum_{i \in r_u} \frac{|\Phi_{ui} - \overline{\Phi^{(i)}}|}{m_i}.$$

- *Weighted Deviation from Mean Agreement (WDMA)* [37]. This feature prioritizes items with fewer ratings, similar to RDMA. The calculation is performed as follows:

$$WDMA_u = \frac{\sum_{i \in r_u} \frac{|\Phi_{ui} - \overline{\Phi^{(i)}}|}{m_i^2}}{|r_u|}.$$

- *Mean Variance (MeanVar)* [37]. This feature calculates the average variance of a user's rating scores compared to the mean rating scores of the corresponding items. The MeanVar

Table 7: Detection results for different attacks.

Attacks	False Negative Rate (FNR)								
	Pantry			Yahoo			ML-1M		
	0.5%	1%	2%	0.5%	1%	2%	0.5%	1%	2%
Random	0.0381	0.0287	0.0425	0.0141	0.0141	0.0144	0.0575	0.0411	0.0462
Bandwagon	0.0405	0.0345	0.0425	0.0240	0.0280	0.0251	0.0261	0.0354	0.0389
MF	0.0429	0.0345	0.0328	0.0318	0.0182	0.0180	0.0310	0.0441	0.0471
DNN	0.0020	0.0080	0.0420	0.0509	0.0482	0.0455	0.0428	0.0524	0.0779
PARL	0.3048	0.2172	0.1195	0.1526	0.1418	0.1418	0.3208	0.2280	0.1667

feature for a user u is computed as follows:

$$\text{MeanVar}_u = \frac{\sum_{i \in \mathbf{r}_u} |\Phi_{ui} - \overline{\Phi^{(i)}}|^2}{|\mathbf{r}_u|}.$$

- *Filler Mean Target Difference (FMTD)* [37]. This feature quantifies the divergence between the rating scores of a user, and it is obtained through the following process:

$$\text{FMTD}_u = \left| \frac{\sum_{i \in \mathbf{r}_{uM}} \Phi_{ui}}{|\mathbf{r}_{uM}|} - \frac{\sum_{j \in \mathbf{r}_{uO}} \Phi_{uj}}{|\mathbf{r}_{uO}|} \right|,$$

where $\mathbf{r}_{uM}$ represents the set of items in $\mathbf{r}_u$ that received the maximum rating score from user u , while $\mathbf{r}_{uO}$ consists of all the other items in $\mathbf{r}_u$.

For each poisoning attack, the service provider generates a certain amount of fake users using the attack and extracts the same number of normal users from the original dataset to form a user set. The training dataset mixes features from 150 fake users and 150 normal users in the experiments. The service provider trains SVM classifiers for each attack separately based on the above features. SVM is suitable for detecting fake users and has been widely adopted by prior works [14, 22] due to two reasons: 1) It can deal with severely skewed-distributed normal users and fake users. 2) It has a nonlinear fitting ability and good interpretability.

We deploy trained SVM classifiers to detect fake users generated by different attacks. Table 7 shows the fake users' detection results of different attacks, where False Negative Rate (FNR) stands for the fraction of the fake users that are misclassified to normal users. 1) PARL has higher FNRs than other attack methods under all settings. For instance, the best FNR of other methods does not exceed 0.08, while the lowest FNR of PARL is more than 0.14. It indicates that PARL is more capable of evading detectors.

2) The FNR of PARL tends to decrease as the attacker injects more fake users. A possible reason is that generating more fake users needs more loop executions of Phase 2 and Phase 3. The poison model tends to be convergent, *i.e.*, the variation between the unoptimized and optimized models becomes small. Thus, the selected filler items fill into a similar pattern.

Table 8: Detection results for PARL with or without order rearrangement.

Dataset	Order Rearrangement	FNR		
		0.5%	1%	2%
Pantry	No	0.3201	0.2756	0.2861
	Yes	0.3209	0.2935	0.3424
ML-1M	No	0.2224	0.2254	0.4489
	Yes	0.2803	0.3515	0.4578

6.2 Sequence Based Detection

Recalling Section 3.2, PARL should keep the sequential feature of the fake users' records aligned with that of the normal users due to the sequential decision-making process of the DRL-based recommender system. Thus, we separately detect the fake users generated with or without order rearrangement in Section 4.4. We align the users' records according to the shortest length of the sequence and train a CNN classifier to divide the normal and the fake users. Since the users' records of the Yahoo dataset do not have timestamps, we provide the FNR results on datasets Pantry and ML-1M. We keep the other settings the same as those of Section 6.1.

We have the following observations from the results of Table 8. 1) The fake users generated by PARL with the order rearrangement are more challenging to distinguish from the normal users. Since the Transformer model learns on the normal users' items in Phase 1, the resorted items of the fake users mimic the sequential properties of the normal users.

2) Without the order rearrangement, all FNR results from PARL are less than 50%. Thus, detecting fake users based on sequence features is valid in RL scenarios. The poisoning attack against the DRL-based recommender system requires recovering sequential features of fake data to evade detection more effectively.

7 RELATED WORK

Poisoning Attacks against DRL Systems. One could argue that poisoning attacks against DNN models could be easily adapted to the DRL setting since it is essentially DNN plus RL. However, DRL differs from DNN due to the sequential decision-making feature,

which is barely considered in existing work. Since PARL integrates the order rearrangement into the poisoning data generation, it is superior to the DNN attack in effectiveness and stealthiness from the experimental results, such as Table 3 and Table 8.

The competitors used in the paper are optimized for the recommender system, which also serves as the baselines in prior works [22, 26]. Several poisoning attacks against DRL, such as [4, 21, 33, 58, 65]. However, they mainly differ from PARL in two aspects. First, existing attacks are performed by modifying the normal users' ratings, while PARL cannot manipulate the normal users. Second, existing work mainly disturbs the recommendation of the DRL model, while PARL is optimized for improving the target item's ranking and being stealthy.

Poisoning Attacks against Recommender Systems. Early poisoning attacks usually did not rely on the characteristics of the target system, *i.e.*, system-untargeted attacks. For example, the random attacks [23, 38] inject fake users into the system and randomly assign ratings to selected items based on a normal distribution calculated from the entire user-item rating matrix. In bandwagon attack [23, 42], the attacker assigns a rating to a selected item based on a normal distribution calculated from the rating scores of that item in the user-item rating matrix. To evaluate the security of recommender systems against these attacks, Mobasher *et al.* [37] took into account the overall impact on the system's ability to make accurate predictions. They also considered the level of knowledge the attacker requires to execute a realistic attack. On the other hand, O'Mahony *et al.* [41] presented a theory for analyzing the robustness of a recommender system, which focuses on the system's resilience to potentially malicious customer or product rating matrix alterations. Several system-targeted poisoning attacks have been proposed recently, *e.g.*, graph-based [15], association-rule-based [59], matrix-factorization-based [14, 26], neighborhood-based [7], and DNN-based recommender systems [22, 44].

Other Attacks against Recommender Systems. Profile pollution attacks [28, 29, 56] aim to deceive or manipulate the recommender system by introducing false or misleading information into the attacker's profile. By doing so, attackers attempt to influence the recommendations to benefit them or undermine the system's performance. Inference attacks [5, 11, 17, 25, 61] leverage the recommendations made by the system to make educated guesses or infer hidden attributes or preferences of specific users. By carefully observing the recommendations received by a target user and combining them with external knowledge or statistical analysis, the attacker attempts to deduce sensitive information that the user may not have explicitly disclosed.

8 DISCUSSION

Highlights of PARL. 1) PARL is the first poisoning attack considering the sequential decision-making characteristics of the DRL-based recommender system. Therefore, PARL reorders the items' sequence of the fake users to replicate the sequential patterns of genuine users, which is often neglected in prior research. 2) Through an exhaustive analysis of PARL across various experimental configurations, including variations in the number of fake users, recommended items, and filler items. We derive valuable insights that advocate for the adoption of PARL. Furthermore, the black-box

setting usually limits the attacker's access to the normal users' records and the model structure based on the analysis of [66]. Thus, in Section 5.3, we show the performance of PARL with limited user records. In Section 5.4, we show the effectiveness of PARL with partial information on the model structure. 3) In Section 6, we demonstrate the robustness against two fake user detectors for the recommender system, *i.e.*, rating score-based detector and sequence-based detector. The defender can mitigate the risks of our attack by eliminating the detected fake users before training the recommender systems. However, at least 11.95% fake users are still misclassified as normal users.

Practical Impacts of PARL. Attacks that poison DRL-based recommender systems, such as PARL, can adversely affect these systems' performance, equity, and reliability. By strategically promoting specific items, such attacks lead to imprecise recommendations that diminish the system's overall quality. For enterprises dependent on recommender systems to stimulate sales or engagement, such attacks can carry financial implications, including reduced revenue and customer discontent. As users encounter recommendations that are evidently flawed or skewed, their confidence in the system may waver. Consequently, this can culminate in reduced interaction, diminished user contentment, and potentially even the system's abandonment.

Limitations and Future Work. Below, we discuss the limitations of PARL and promising directions for further improvements. 1) From Section 4.2, PARL only generates a fake user after three sequentially executed phases. Thus, it is interesting to improve PARL's effectiveness by parallelization. 2) All current approaches, including PARL, exhibit limited effectiveness when dealing with a small fraction of fake users. Hence, incorporating the count of fake users as an optimization objective could potentially offer a more promising avenue for advancement.

Defenses against PARL. To address the ethical risks of PARL, we explore two defense methods in Section 6. However, the experimental results demonstrate that recovering the sequence of the records can improve the detection escape rate of fake users.

There are two possible ways to improve defenses. Firstly, the DRL-based recommender systems can be trained using proxy datasets instead of original datasets, *e.g.*, data augmentation [3, 6, 12, 63, 70] and data synthesis methods based on differential privacy [13, 51, 52, 54, 62, 67, 68]. They slightly alter the original dataset's distribution, decreasing the attacks' effectiveness. Secondly, defenders can monitor the recommended content over time to detect abnormalities in the output of the DRL-based recommender system.

9 CONCLUSION

In this paper, we design PARL to launch poisoning attacks against DRL-based recommender systems. The insight behind PARL involves enhancing the ranking of the target item while keeping the rankings of other items unchanged to maximize the hit ratio while evading detection. The hit ratio increased by 70× at most compared to the normal case with only 1% fake users, demonstrating that PARL is an effective strategy to manipulate the behavior of the recommender system. With only partial knowledge of normal users' ratings, the hit ratio of PARL still surpasses the competitors by at least 1.8×. Additionally, our investigation of the defense methods

reveals the potential of sequence features in differentiating between normal users and fake users, which has been frequently overlooked in previous poisoning attacks.

ACKNOWLEDGEMENT

We express our gratitude to the anonymous reviewers for their valuable feedback. This work was partly supported by the NSFC under Grants 61833015, 62293511, 62088101, 52161135201, 62103371, the Fundamental Research Funds for the Central Universities 226-2023-00111, 226-2024-00004. Min Chen was sponsored by the Helmholtz Association within the project “Trustworthy Federated Data Analytics” (TFDA) (No. ZT-I-001 4).

REFERENCES

- [1] M. M. Afsar, T. Crump, and B. Far. Reinforcement Learning Based Recommender Systems: A Survey. *ACM Computing Surveys*, 2022.
- [2] B. Biggio, B. Nelson, and P. Laskov. Poisoning Attacks against Support Vector Machines. In *ICML*, 2012.
- [3] E. Borgnia, V. Cherepanova, L. H. Fowl, A. Ghiasi, J. Geiping, M. Goldblum, T. Goldstein, and A. Gupta. Strong Data Augmentation Sanitizes Poisoning and Backdoor Attacks Without an Accuracy Tradeoff. In *ICASSP*, 2020.
- [4] K. Cai, X. Zhu, and Z. Hu. Reward Poisoning Attacks in Deep Reinforcement Learning Based on Exploration Strategies. *Neurocomputing*, 2023.
- [5] J. A. Calandrino, A. Kilzer, A. Narayanan, E. W. Felten, and V. Shmatikov. You Might Also Like: Privacy Risks of Collaborative Filtering. In *IEEE S&P*, 2011.
- [6] J. Chen, X. Zhang, R. Zhang, C. Wang, and L. Liu. De-Pois: An Attack-Agnostic Defense against Data Poisoning Attacks. *IEEE TIFS*, 2021.
- [7] L. Chen, Y. Xu, F. Xie, M. Huang, and Z. Zheng. Data Poisoning Attacks on Neighborhood-based Recommender Systems. *Transactions on Emerging Telecommunications Technologies*, 2021.
- [8] M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, and E. H. Chi. Top-k Off-policy Correction for a REINFORCE Recommender System. In *WSDM*, 2019.
- [9] X. Chen, L. Yao, J. J. McAuley, G. Zhou, and X. Wang. Deep Reinforcement Learning in Recommender Systems: A Survey and New Perspectives. *Knowledge-based Systems*, 2023.
- [10] P.-A. Chirita, W. Nejdl, and C. Zamfir. Preventing Shilling Attacks in Online Recommender Systems. In *WIDM*, 2005.
- [11] L. Du, M. Chen, M. Sun, S. Ji, P. Cheng, J. Chen, and Z. Zhang. ORL-Auditor: Dataset Auditing in Offline Deep Reinforcement Learning. In *NDSS*, 2024.
- [12] L. Du, Z. Zhang, S. Bai, C. Liu, S. Ji, P. Cheng, and J. Chen. AHEAD: Adaptive Hierarchical Decomposition for Range Query under Local Differential Privacy. In *ACM CCS*, 2021.
- [13] Y. Du, Y. Hu, Z. Zhang, Z. Fang, L. Chen, B. Zheng, and Y. Gao. LDPTTrace: Locally Differentially Private Trajectory Synthesis. In *Vldb*, 2023.
- [14] M. Fang, N. Z. Gong, and J. Liu. Influence Function Based Data Poisoning Attacks to Top-n Recommender Systems. In *WWW*, 2020.
- [15] M. Fang, G. Yang, N. Z. Gong, and J. Liu. Poisoning Attacks to Graph-based Recommender Systems. In *ACSAC*, 2018.
- [16] M. Fu, L. Huang, A. Rao, A. A. Irissappane, J. Zhang, and H. Qu. A Deep Reinforcement Learning Recommender System With Multiple Policies for Recommendations. *IEEE TII*, 2023.
- [17] N. Z. Gong and B. Liu. You Are Who You Know and How You Behave: Attribute Inference Attacks via Users’ Social Friends and Behaviors. In *USENIX Security*, 2016.
- [18] K. Hao. How Facebook Uses Machine Learning to Detect Fake Accounts, Apr 2020.
- [19] F. M. Harper and J. A. Konstan. The MovieLens Datasets: History and Context. *ACM TIS*, 2015.
- [20] N. Heess, D. TB, S. Sriram, J. Lemmon, J. Merel, G. Wayne, Y. Tassa, T. Erez, Z. Wang, S. M. A. Eslami, M. A. Riedmiller, and D. Silver. Emergence of Locomotion Behaviours in Rich Environments. *CoRR*, abs/1707.02286, 2017.
- [21] M. Huai, J. Sun, R. Cai, L. Yao, and A. Zhang. Malicious Attacks against Deep Reinforcement Learning Interpretations. In *SIGKDD*, 2020.
- [22] H. Huang, J. Mu, N. Z. Gong, Q. Li, B. Liu, and M. Xu. Data Poisoning Attacks to Deep Learning Based Recommender Systems. In *NDSS*, 2021.
- [23] S. Kapoor, V. Kapoor, and R. Kumar. A Review of Attacks and Its Detection Attributes on Collaborative Recommender Systems. *IJARCS*, 2017.
- [24] V. R. Konda and J. N. Tsitsiklis. Actor-Critic Algorithms. In *NeuIPS*, 1999.
- [25] M. Kosinski, D. Stillwell, and T. Graepel. Private Traits and Attributes are Predictable from Digital Records of Human Behavior. *PNAS*, 2013.
- [26] B. Li, Y. Wang, A. Singh, and Y. Vorobeychik. Data Poisoning Attacks on Factorization-based Collaborative Filtering. In *NeuIPS*, 2016.
- [27] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous Control with Deep Reinforcement Learning. *CoRR*, abs/1509.02971, 2015.
- [28] C. Lin, S. Chen, H. Li, Y. Xiao, L. Li, and Q. Yang. Attacking Recommender Systems with Augmented User Profiles. In *CIKM*, 2020.
- [29] C. Lin, S. Chen, M. Zeng, S. Zhang, M. Gao, and H. Li. Shilling Black-box Recommender Systems by Learning to Generate Fake User Profiles. *CoRR*, abs/2206.11433, 2022.
- [30] Y.-C. A. Lin, Y. Liu, F. Lin, P. Wu, W. Zeng, and C. Miao. A Survey on Reinforcement Learning for Recommender Systems. *IEEE TNNLS*, 2021.
- [31] F. Liu, H. Guo, X. Li, R. Tang, Y. Ye, and X. He. End-to-End Deep Reinforcement Learning Based Recommendation with Supervised Embedding. In *WSDM*, 2020.
- [32] F. Liu, R. Tang, X. Li, Y. Ye, H. Chen, H. Guo, and Y. Zhang. Deep Reinforcement Learning based Recommendation with Explicit User-Item Interactions Modeling. *CoRR*, abs/1810.12027, 2018.
- [33] O. Ma, Y. Pu, L. Du, Y. Dai, R. Wang, X. Liu, Y. Wu, and S. Ji. SUB-PLAY: Adversarial Policies against Partially Observed Multi-Agent Reinforcement Learning Systems. *CoRR*, abs/2402.03741, 2024.
- [34] B. M. Marlin and R. S. Zemel. Collaborative Prediction and Ranking with Non-Random Missing Data. In *ACM RecSys*, 2009.
- [35] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous Methods for Deep Reinforcement Learning. In *ICML*, 2016.
- [36] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller. Playing Atari with Deep Reinforcement Learning. *CoRR*, abs/1312.5602, 2013.
- [37] B. Mobasher, R. Burke, R. Bhaumik, and C. Williams. Toward Trustworthy Recommender Systems: An Analysis of Attack Models and Algorithm Robustness. *ACM TOIT*, 2007.
- [38] B. Mobasher, R. D. Burke, R. Bhaumik, and J. J. Sandvig. Attacks and Remedies in Collaborative Recommendation. *IEEE Intelligent Systems*, 2007.
- [39] T. Nguyen Thanh, N. D. K. Quach, T. T. Nguyen, T. T. Huynh, V. H. Vu, P. L. Nguyen, J. Jo, and Q. V. H. Nguyen. Poisoning GNN-based Recommender Systems with Generative Surrogate-based Attacks. *ACM TIS*, 2023.
- [40] J. Ni, J. Li, and J. McAuley. Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects. In *EMNLP-IJCNLP*, 2019.
- [41] M. P. O’Mahony, N. J. Hurley, N. Kushmerick, and G. C. M. Silvestre. Collaborative Recommendation: A Robustness Analysis. *ACM TOIT*, 2004.
- [42] M. P. O’Mahony, N. J. Hurley, and G. C. M. Silvestre. Recommender Systems: Attack Types and Strategies. In *AAAI*, 2005.
- [43] H. Pi, Z. Ji, and C. Yang. A Survey of Recommender System from Data Sources Perspective. In *MEICI*, 2018.
- [44] H. Pu, L. He, P. Cheng, M. Sun, and J. Chen. Security of Industrial Robots: Vulnerabilities, Attacks, and Mitigations. *IEEE Network*, 2023.
- [45] D. Rong, Q. He, and J. Chen. Poisoning Deep Learning based Recommender Model in Federated Learning Scenarios. *CoRR*, abs/2204.13594, 2022.
- [46] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *CoRR*, abs/1707.06347, 2017.
- [47] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. P. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the Game of Go without Human Knowledge. *Nature*, 2017.
- [48] G. Sun, Y. Cong, J. Dong, Q. Wang, and J. Liu. Data Poisoning Attacks on Federated Machine Learning. *IEEE IoTJ*, 2020.
- [49] G. Tytunovich. Council Post: The Great Contamination: How Bots and Fake Users can Skew an Organization’s Data and Analytics, Jan 2022.
- [50] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is All you Need. In *NeuIPS*, 2017.
- [51] H. Wang, Z. Zhang, T. Wang, S. He, M. Backes, J. Chen, and Y. Zhang. PrivTrace: Differentially Private Trajectory Synthesis by Adaptive Markov Model. In *USENIX Security*, 2023.
- [52] T. Wang, J. Q. Chen, Z. Zhang, D. Su, Y. Cheng, Z. Li, N. Li, and S. Jha. Continuous Release of Data Streams under both Centralized and Local Differential Privacy. In *ACM CCS*, 2021.
- [53] C. J. C. H. Watkins. Learning from Delayed Rewards. *PhD Thesis, Cambridge University, Cambridge, England*, 1989.
- [54] C. Wei, M. Zhao, Z. Zhang, M. Chen, W. Meng, B. Liu, Y. Fan, and W. Chen. DPMLBench: Holistic Evaluation of Differentially Private Machine Learning. In *ACM CCS*, 2023.
- [55] R. J. Williams. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 1992.
- [56] X. Xing, W. Meng, D. Doozan, A. C. Snoeren, N. Feamster, and W. Lee. Take This Personally: Pollution Attacks on Personalized Services. In *USENIX Security*, 2013.
- [57] T. Xu, Q. Liu, L. Zhao, and J. Peng. Learning to Explore via Meta-Policy Gradient. In *ICML*, pages 5459–5468, 2018.
- [58] Y. Xu, Q. Zeng, and G. Singh. Efficient Reward Poisoning Attacks on Online Deep Reinforcement Learning. *CoRR*, abs/2205.14842, 2022.
- [59] G. Yang, N. Z. Gong, and Y. Cai. Fake Co-visitation Injection Attacks to Recommender Systems. In *NDSS*, 2017.

- [60] Z. Yang, L. He, H. Yu, C. Zhao, P. Cheng, and J. Chen. Detecting PLC Intrusions Using Control Invariants. *IEEE Internet of Things Journal*, 9, 2022.
- [61] Z. Yang, L. He, H. Yu, C. Zhao, P. Cheng, and J. Chen. Reverse Engineering Physical Semantics of PLC Program Variables Using Control Invariants. In *ACM SenSys*, 2022.
- [62] Q. Yuan, Z. Zhang, L. Du, M. Chen, P. Cheng, and M. Sun. PrivGraph: Differentially Private Graph Data Publication by Exploiting Community Information. In *USENIX Security*, 2023.
- [63] Y. Zeng, H. Qiu, S. Guo, T. Zhang, M. Qiu, and B. M. Thuraisingham. DeepSweep: An Evaluation Framework for Mitigating DNN Backdoor Attacks using Data Augmentation. In *ACM ASIACCS*, 2020.
- [64] M. Zhang, Z. McCarthy, C. Finn, S. Levine, and P. Abbeel. Learning Deep Neural Network Policies with Continuous Memory States. In *ICRA*, 2016.
- [65] X. Zhang, Y. Ma, A. Singla, and X. Zhu. Adaptive Reward-Poisoning Attacks against Reinforcement Learning. In *ICML*, 2020.
- [66] Y. Zhang, X. Yuan, J. Li, J. Lou, L. Chen, and N.-F. Tzeng. Reverse Attack: Black-box Attacks on Collaborative Recommendation. In *ACM CCS*, 2021.
- [67] Z. Zhang, T. Wang, N. Li, S. He, and J. Chen. CALM: Consistent Adaptive Local Marginal for Marginal Release under Local Differential Privacy. In *ACM CCS*, 2018.
- [68] Z. Zhang, T. Wang, N. Li, J. Honorio, M. Backes, S. He, J. Chen, and Y. Zhang. PrivSyn: Differentially Private Data Synthesis. In *USENIX Security*, 2021.
- [69] G. Zheng, F. Zhang, Z. Zheng, Y. Xiang, N. J. Yuan, X. Xie, and Z. Li. DRN: A Deep Reinforcement Learning Framework for News Recommendation. In *WWW*, 2018.
- [70] H. Zhu, M. Liu, C. Fang, R. Deng, and P. Cheng. Detection-Performance Tradeoff for Watermarking in Industrial Control Systems. *IEEE TIFS*, 2023.